

Running this script produces the following output. Notice how the sequence length (the middle dimension) of the logit tensors decreases by one at each step, confirming our causal chain is correctly implemented.

```
Model created with 2.01M params
```

```
Logits shapes:
```

```
Main      : [2, 20, 1000]
MTP k=1   : [2, 19, 1000]
MTP k=2   : [2, 18, 1000]
MTP k=3   : [2, 17, 1000]
```

```
Total loss: 109.1470
```

With this, we have successfully implemented the core logic of DeepSeek's Multi-Token Prediction architecture. We've seen how a causal chain of dedicated Transformer blocks can be used to predict a horizon of future tokens, providing a richer training signal and a powerful mechanism for better planning and forecasting.

5.5 Quantization: Trading precision for speed and memory

We have now completed our exploration of the high-level architectural pillars of the DeepSeek model: Multi-Head Latent Attention, Mixture of Experts, and Multi-Token Prediction. These innovations define *what* the model computes. Now, we turn our attention to an equally important topic that defines *how* these computations are performed with incredible efficiency: FP8 Quantization.

This topic is the final key to understanding how DeepSeek manages to train and run massive, 671-billion-parameter models at a fraction of the cost of its competitors. As we will see, their approach to quantization is a sophisticated, multi-faceted strategy that pushes the boundaries of low-precision training. While "low precision" might sound like a compromise, it is the essential trade-off that unlocks incredible gains in speed and memory efficiency, making large-scale training possible.

We will start by building a solid foundation, understanding what quantization is and why it's necessary. Then, we will deconstruct the five major innovations that make up DeepSeek's FP8 training framework.

5.5.1 What is quantization?

At its core, every parameter in a large language model, every weight in every matrix, every bias in every layer is just a number. By default, computers store these numbers with a high degree of precision, typically using a format called 32-bit floating-point (FP32).

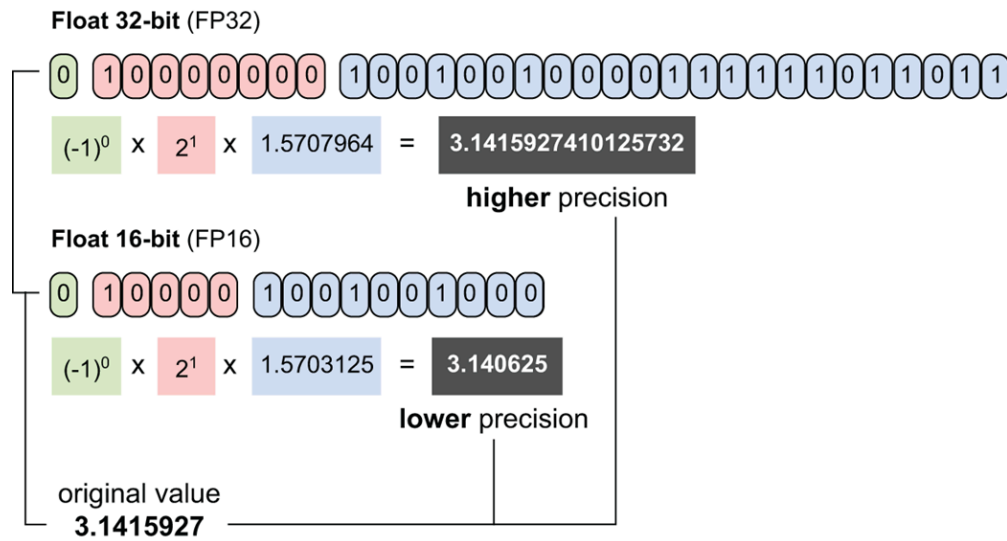


Figure 5.15 A visual comparison of a number represented in 32-bit floating-point (FP32) versus 16-bit floating-point (FP16). The diagram illustrates the reduction in the number of bits allocated for the exponent and the mantissa, which results in lower precision and a smaller memory footprint.

As shown in figure 5.15, an FP32 number uses 32 bits of memory to represent a single value. This allows for very high precision (many decimal places) and a vast dynamic range (the ability to represent both very large and very small numbers).

Quantization is the process of reducing this precision. It is a technique for converting a model's parameters from a higher bit-width to a lower bit-width. For example, we might quantize a model from FP32 to FP16, meaning every parameter now only uses 16 bits of memory instead of 32.

The intuition behind this is best understood with an analogy.

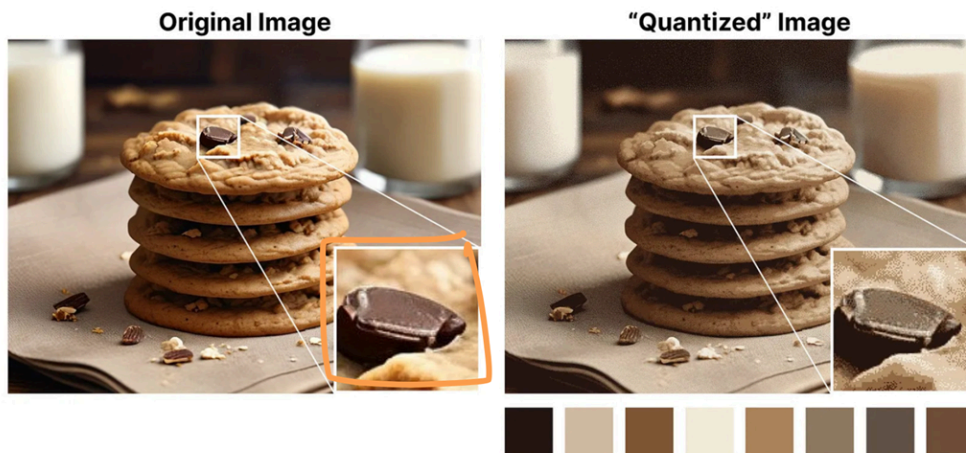


Figure 5.16 The quantized image uses far fewer colors (less information/precision) but still effectively represents the original image.

As illustrated in figure 5.16, the original image uses a vast palette of colors to represent every detail with perfect fidelity. The quantized image uses only a small, limited palette of 8 colors. While a close-up inspection reveals a loss of detail (pixelation), the overall image is still clearly recognizable.

Quantization does the same thing to a neural network. It reduces the "palette" of numbers the model can use. While this results in a slight loss of precision for each individual parameter, the overall performance of the model often remains remarkably robust.

5.5.2 Why quantize? The memory cost of high-precision parameters

The primary motivation for quantization is to solve the enormous memory and computational costs associated with large language models. Storing billions of parameters in high precision is incredibly expensive.

$$\mathbf{64\text{-bits}} = \frac{64}{8} \times 70\text{B} \approx \mathbf{560\text{ GB}}$$

.....

$$\mathbf{32\text{-bits}} = \frac{32}{8} \times 70\text{B} \approx \mathbf{280\text{ GB}}$$

.....

$$\mathbf{16\text{-bits}} = \frac{16}{8} \times 70\text{B} \approx \mathbf{140\text{ GB}}$$

Figure 5.17 The memory savings from quantization for a 70-billion parameter model. The calculations demonstrate how reducing the numerical precision from 64-bits to 32-bits, and further to 16-bits, dramatically decreases the total memory required to store the model's weights.

As figure 5.17 shows, the memory savings are dramatic. By quantizing a 70B parameter model from 32-bit to 16-bit, we cut its memory requirement in half, from a staggering 280 GB down to a more manageable 140 GB. This reduction in memory has two direct benefits:

1. **Faster Training and Inference:** Smaller parameters mean less data needs to be moved from the GPU's main memory to its compute cores, which significantly speeds up every calculation.
2. **Accessibility:** It allows larger models to be run on hardware with less memory.

This is the central bargain of quantization: we trade a small, often acceptable, amount of precision for a massive gain in memory efficiency and computational speed.

5.5.3 Understanding numerical formats: The building blocks of quantization

To understand the specifics of DeepSeek's framework, we first need to be familiar with the different "palettes" of numbers, or numerical formats, that are commonly used in deep learning. Every floating-point number is represented in memory using three distinct parts:

- **Sign (1 bit):** The simplest part. This single bit determines if the number is positive (0) or negative (1).

- **Exponent:** These bits determine the number's magnitude or dynamic range. They control how large or small the number can be by defining the position of the decimal point (in binary). More exponent bits mean the format can represent a vastly wider range of numbers (e.g., from very close to zero to extremely large).
- **Mantissa (or Significand):** These bits determine the number's precision. They represent the actual digits of the number. More mantissa bits mean the format can store more significant digits, resulting in higher precision and smaller gaps between representable numbers.

Let's see how these components are balanced in the most common formats.

FP32 (32-bit Floating-Point)

This is our high-precision baseline. It uses 1 sign bit, 8 exponent bits, and 23 mantissa bits. Its massive number of mantissa bits gives it very high precision, and the 8 exponent bits give it a vast dynamic range.

FP16 (16-bit Floating-Point)

This was one of the first popular formats for reducing memory. It uses 1 sign bit, 5 exponent bits, and 10 mantissa bits.

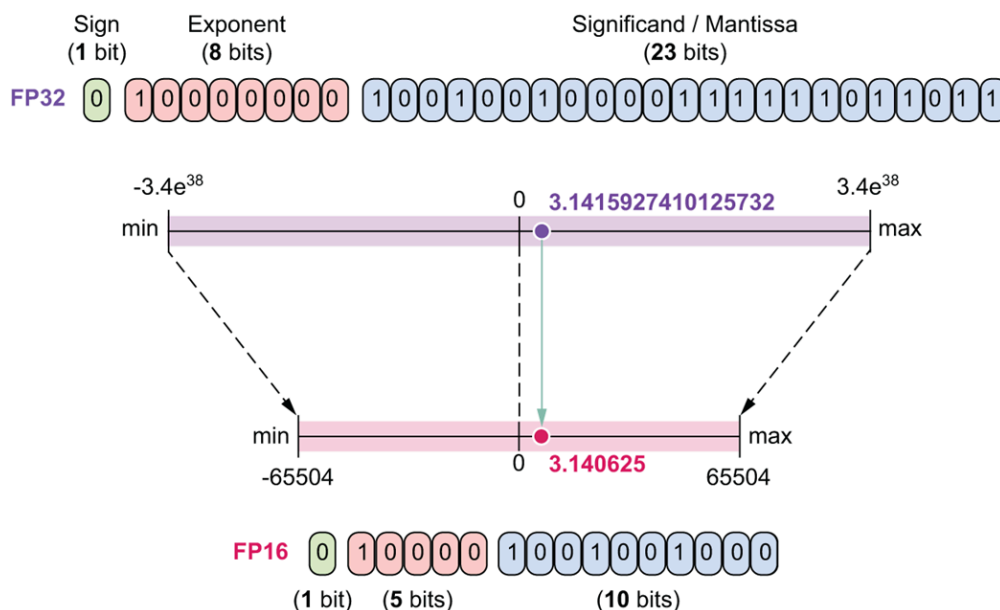


Figure 5.18 A comparison of FP32 and FP16.

As shown in figure 5.18, FP16 drastically reduces the number of bits for both the exponent and the mantissa. As a result, both its range and precision are significantly smaller than FP32. While memory-efficient, it can sometimes suffer from overflow (numbers becoming too large for its limited exponent) or underflow (numbers becoming too small and losing detail).

BF16 (16-bit "Brain Float")

This is a clever format developed by Google to get the best of both worlds for training. It uses 1 sign bit, 8 exponent bits, and 7 mantissa bits.

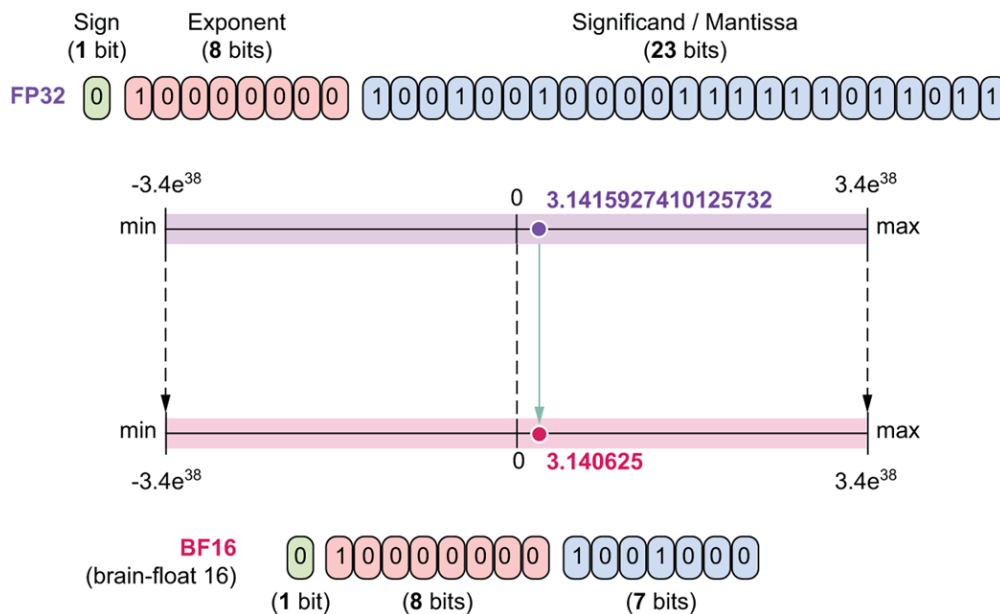


Figure 5.19 A comparison of FP32 and BFloat16.

As figure 5.19 shows, BF16 uses the same 8 exponent bits as FP32, giving it the same massive dynamic range. It saves memory by reducing the mantissa to just 7 bits, which means it has lower precision than even FP16. This format is excellent for training because its wide range makes it very resistant to overflow issues.

INT8 (8-bit Integer)

This is an even more aggressive quantization format. It uses 1 sign bit and 7 bits for the value, with no decimal precision.

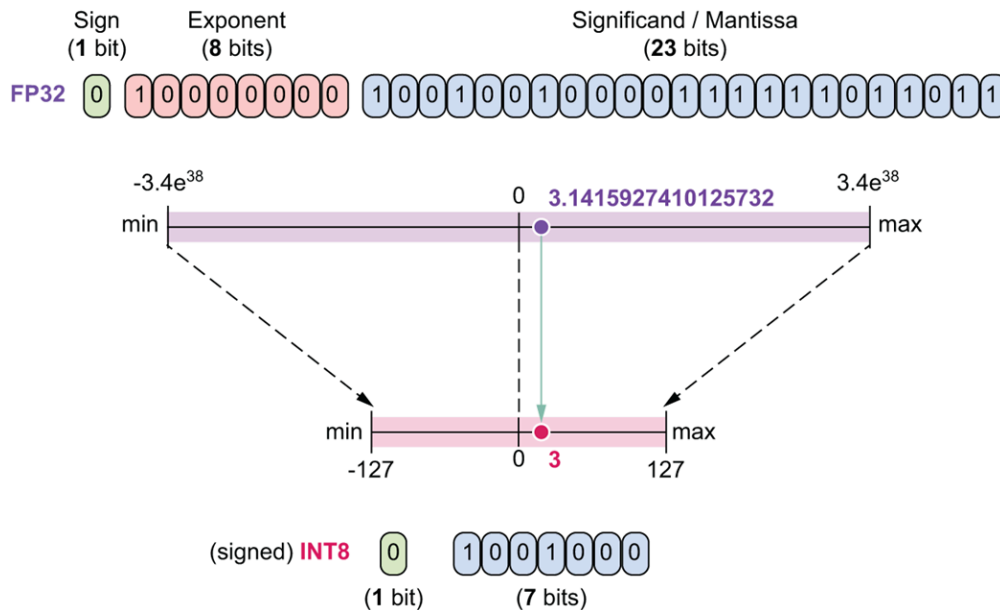


Figure 5.20 A comparison of FP32 and INT8.

As figure 5.20 illustrates, its range is extremely small, from -127 to 127. While highly efficient, converting to INT8 can lead to a more significant loss of information if the original numbers have a wide range.

FP8 (8-bit Floating-Point)

Finally, the format at the heart of DeepSeek's strategy is FP8. It's an 8-bit format that, unlike INT8, still retains a sign, exponent, and mantissa (e.g., E4M3 with 4 exponent bits and 3 mantissa bits). It offers a compromise between the extreme efficiency of INT8 and the flexibility of floating-point numbers.

5.5.4 The basic mechanism: Scaling

How do we actually convert a vector of high-precision FP32 numbers into a lower-precision format like INT8? The process is called scaling. The core idea is to map the range of our original numbers onto the target range of the new format without losing the relative relationships between them.

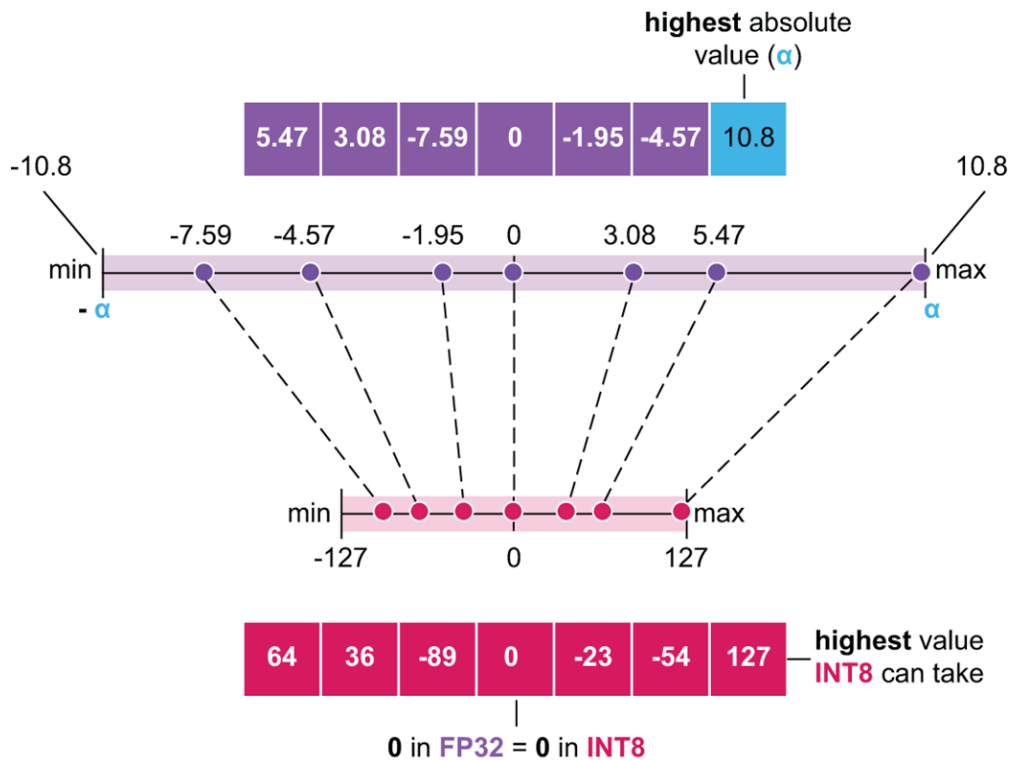


Figure 5.21 The scaling process for quantization. The original range of the FP32 tensor is mapped to the target range of the INT8 format.

As illustrated in figure 5.21, the process involves a few simple steps:

1. **Find the Maximum Absolute Value (α):** We first scan our entire input vector (or tensor) and find the number with the largest absolute value. In this example, it's 10.8. This value, α, defines the effective range of our original data, from -α to +α.
2. **Calculate the Scaling Factor:** We want to map this original range to the target range of the INT8 format, which is -127 to 127. The scaling factor is simply $\text{target_range_max} / \text{original_range_max}$, which is $127 / 10.8$.
3. **Quantize:** We multiply every number in our original vector by this scaling factor and then round to the nearest integer. For example, the number -7.59 becomes $\text{round}(-7.59 * (127 / 10.8))$, which results in -89. This new integer is the quantized representation.
4. **De-quantize:** To recover the original numbers for use in a computation (with some precision loss), we would simply perform the inverse operation: divide the quantized integer by the same scaling factor. For example, $-89 / (127 / 10.8)$ gives us approximately -7.59.

This concept of finding a scaling factor based on the maximum value in a tensor is the most important prerequisite for understanding the advanced techniques used by DeepSeek. As we will see, their first major innovation, fine-grained quantization, is a clever new way of applying this fundamental scaling principle.

5.5.5 The five pillars of DeepSeek's FP8 training

Now that we have a solid foundational understanding of what quantization is and the different numerical formats involved, we can dive into the specifics of DeepSeek's implementation. Their approach is not a single technique but a sophisticated, multi-faceted framework composed of five key innovations that work together to enable stable and efficient training in the ultra-low FP8 precision.

These five pillars are:

1. The Mixed Precision Framework
2. Fine-Grained Quantization
3. Increasing Accumulation Precision
4. Mantissa Over Exponents
5. Online Quantization

Let's deconstruct each of these pillars one by one, explaining the problem they solve and how they are implemented, both visually and mathematically.

5.5.6 Pillar 1: The mixed precision framework

The first and most fundamental pillar of DeepSeek's strategy is the Mixed Precision Framework.

The Core Idea: Not All Numbers are Created Equal

The central insight behind mixed precision is that not all operations or stored values in a neural network require the same level of numerical precision. It would be inefficient to use a 32-bit number with many decimal places for a value that doesn't need it, just as it would be a mistake to use a low-precision number for a value that is highly sensitive to small errors.

A mixed precision framework, therefore, is a strategic system that uses different numerical formats for different parts of the training process. The goal is to get the best of both worlds:

- Use **low-precision formats (like FP8)** for the vast majority of computations (like massive matrix multiplications) to maximize speed and minimize memory usage.
- Use **high-precision formats (like FP32)** for the most sensitive and critical components (like updating the model's master weights) to ensure the training process remains stable and accurate.

DeepSeek's framework is a masterclass in intelligently balancing these trade-offs. Let's walk through how it handles the different operations in a standard linear layer during a full forward and backward pass.

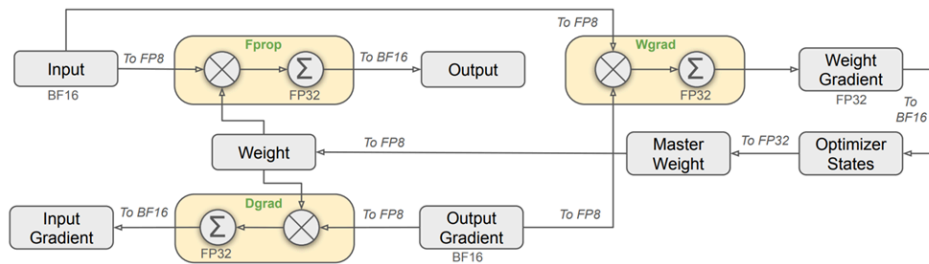


Figure 5.22 The mixed precision framework with FP8 data format in the Linear operator. The diagram illustrates the flow of data through a linear layer, showing how inputs and weights are converted to low-precision FP8 for fast computation, while critical components like master weights and gradients are maintained in high-precision FP32 for stability.

This diagram looks complex, but it simply visualizes the flow of data through the four key stages of a neural network layer's computation. Let's break down each stage.

A. Forward Propagation (Fprop)

The forward pass is the standard prediction step, where $\text{output} = \text{weights} * \text{input}$. This is the primary computation that happens during inference.

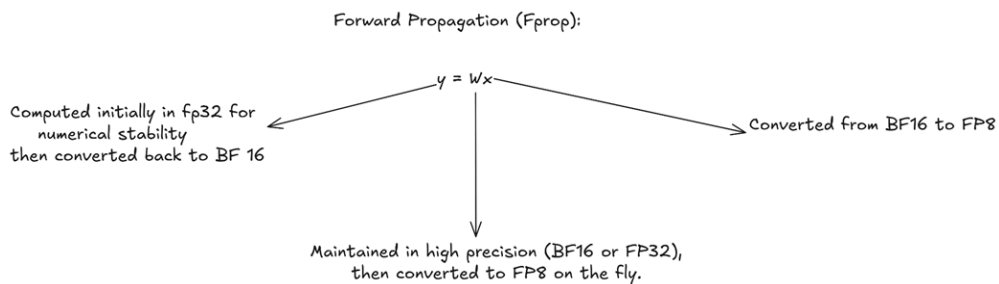


Figure 5.23 The data flow and precision formats for the forward pass.

As shown in figure 5.23, a strategic mix of precisions is used:

- **Input (x):** The input activations from the previous layer are typically stored in **BF16**. For the actual multiplication, they are converted on-the-fly to the highly efficient **FP8** format.

- **Weights (W):** The main "master" copy of the weights is maintained in high-precision **FP32** (or BF16). For the multiplication, these weights are also converted on-the-fly to **FP8**.
- **Output (y):** The result of the FP8 x FP8 multiplication is accumulated in full FP32 to prevent numerical errors and maintain stability. This high-precision result is then immediately converted back down to BF16 for storage in memory.

Why this mix? The heavy matrix multiplication is done in the fastest, lowest-precision format (FP8), while the final result is accumulated and stored in higher-precision formats to prevent the loss of information.

B. Backward Propagation: Gradient with respect to Inputs (Dgrad)

After the forward pass, the model calculates its error, and the learning process begins. Backward propagation involves calculating the gradients of the signals that tell each weight how to update itself to reduce the error. This process is more complex and involves calculating two primary gradients for our $y = Wx$ layer: the gradient with respect to the inputs (dgrad) and the gradient with respect to the weights (wgrad).

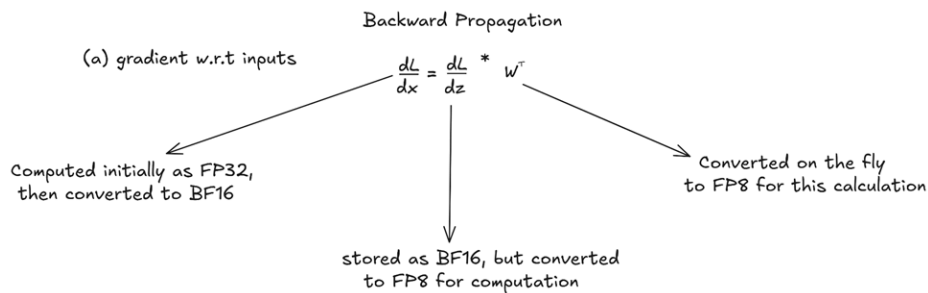


Figure 5.24 The data flow for the Dgrad calculation.

The gradient with respect to the input, dL/dx , is needed to continue the backpropagation process to the previous layer. It's calculated using the chain rule:

$$dL/dx = (dL/dz) * W^T$$

Here, dL/dz is the gradient coming from the next layer, and W^T is the transpose of the weight matrix. DeepSeek applies a similar mixed precision strategy here:

- The incoming gradient (dL/dz), stored as BF16, is converted to FP8 for the computation.
- The original weight matrix (W), which is in high precision, is also converted on the fly to FP8 for this calculation.
- The resulting gradient, dL/dx , is computed with an FP32 accumulator and then stored as BF16 to be passed back to the previous layer.

Notice the symmetry with the forward pass: the core operation is FP8 for speed, but the result that gets passed between layers is kept in the more stable BF16 format.

C. Backward Propagation: Gradient with respect to Weights (Wgrad)

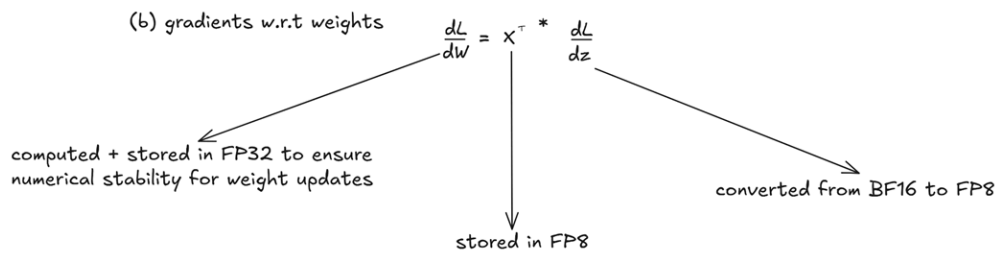


Figure 5.25 The data flow for the Wgrad calculation.

This is the most critical gradient, as it will be used to update the model's actual knowledge. The gradient dL/dW tells the model how to adjust its weights. It is calculated as:

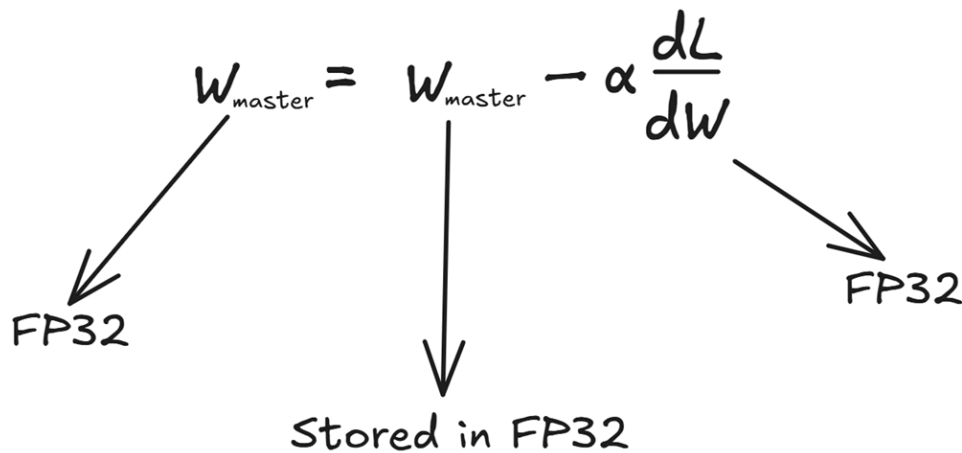
$$dL/dW = X^T * (dL/dz)$$

Here, the precision strategy changes to prioritize accuracy. A noisy or imprecise weight gradient can destabilize the entire training process. Therefore, DeepSeek makes a crucial decision:

- The input to the layer (x), which was stored in FP8 during the forward pass, is used here.
- The incoming gradient (dL/dz) is converted from BF16 to FP8.
- The resulting weight gradient, dL/dW , is computed and, most importantly, stored in full FP32 precision.

This is a key part of the "mixed" precision framework. While other gradients can be stored in BF16, the weight gradient is kept at the highest fidelity to ensure that the updates to the model's core parameters are as accurate as possible.

D. The Weight Update



After updating, master weights are converted to BF16 for next iteration forward pass or FP8 conversion when required.

Figure 5.26 The weight update step is performed entirely in high-precision FP32.

Finally, the optimizer uses the high-precision weight gradient to update the master weights.

$$W_{\text{master_new}} = W_{\text{master_old}} - \text{learning_rate} * (dL/dW)$$

Because this step directly modifies the model's permanent knowledge, stability is paramount. Therefore, all components of this operation are maintained in FP32:

- The **master weights** are stored in FP32.
- The **weight gradients** (dL/dW) are already in FP32.
- The optimizer's internal states (like momentum and variance in AdamW) are also kept in FP32.

The entire update happens in high precision. After the $W_{\text{master_new}}$ is computed, this FP32 version is stored. For the *next* training iteration's forward pass, these master weights will once again be converted on the fly to FP8, completing the cycle.

By converting inputs and weights to FP8 for all three major GEMM (General Matrix Multiplication) operations (Fprop, Dgrad, Wgrad), DeepSeek maximizes the throughput and leverages the full power of modern hardware. This provides up to a 2x speed improvement over BF16 operations.

They identified the most sensitive parts of the training loop. The master weights and optimizer states are kept in FP32 to prevent error accumulation and ensure stable learning. Inter-layer activations and gradients (y and dL/dx) are kept in BF16, providing a robust middle ground that is more memory-efficient than FP32 but safer than FP8.

DeepSeek's team went even further. They identified that certain modules in the Transformer architecture are more sensitive to quantization errors than others. As a result, they chose to keep these specific modules in higher precision (BF16), completely bypassing the FP8 quantization for them. These sensitive components include:

- The Embedding Modules (both token and positional)
- The final Output Head (which projects to the vocabulary)
- Mixture-of-Experts (MoE) Gating Modules
- Normalization Layers (e.g., RMSNorm)
- Attention Operators (specifically, the softmax and context vector calculation)

This balance of high-speed, low-precision computation with high-precision storage for critical components allows DeepSeek to train massive models faster and with less memory, without succumbing to the instability that plagues naive low-precision training approaches. It is the bedrock upon which the other four pillars of their FP8 framework are built.

5.5.7 Pillar 2: Fine-grained quantization

The Mixed Precision Framework defines which numerical formats to use for specific operations within the model. The second pillar, Fine-Grained Quantization, addresses how to convert numbers from a high-precision format like BF16 to a low-precision format like FP8 in a way that preserves as much information as possible.

As we covered in section 5.5.4, the standard mechanism for this conversion is scaling. We find the maximum absolute value in an entire tensor, and we use that value to scale every single number in the tensor into the target range (e.g., the range of FP8), but it does come with one problem.

THE PROBLEM WITH OUTLIERS IN STANDARD QUANTIZATION

This standard, tensor-wise scaling approach has a major weakness: it is extremely sensitive to outliers. Even a single large value in a massive tensor can drastically reduce the precision for all other values.

Let's illustrate this with a concrete numerical example. Imagine we have a small vector of activation outputs that we want to quantize to an 8-bit integer format (range -127 to 127): [2.0, 3.0, 500.0]. The presence of the 500.0 outlier completely corrupts the precision of the other values:

1. **Find Max Absolute Value (a):** The maximum absolute value is $a = 500.0$.
2. **Calculate Scaling Factor (s):** To map the range $[-500, 500]$ to $[-127, 127]$, our scaling factor is $s = 127 / 500 = 0.254$.
3. **Quantize:** We multiply each element by s and round to the nearest integer:
 - $\text{round}(2.0 * 0.254) = \text{round}(0.508) = 1$
 - $\text{round}(3.0 * 0.254) = \text{round}(0.762) = 1$
 - $\text{round}(500.0 * 0.254) = \text{round}(127.0) = 127$

The resulting quantized vector is $[1, 1, 127]$. The crucial distinction between 2.0 and 3.0 has been completely erased.
4. **De-quantize:** When we convert back by dividing by s , we get $[1/0.254, 1/0.254, 127/0.254]$, which is approximately $[3.94, 3.94, 500.0]$.

The relative error for the first two values is catastrophic (97% and 31% respectively), while the outlier is recovered almost perfectly. This is a critical problem in large language models, where activation values can have enormous dynamic ranges.

THE DEEPSEEK SOLUTION: GROUPING FOR SEPARATE SCALING

To solve this, DeepSeek implemented a technique called Fine-Grained Quantization. The idea is brilliantly simple: if a single scaling factor for the whole tensor is problematic, why not break the tensor into smaller chunks and use a separate scaling factor for each chunk?

Instead of calculating one maximum value for the entire tensor, DeepSeek divides the tensor into smaller chunks (or "groups" or "tiles") and calculates a separate scaling factor for each chunk independently. This strategy is applied differently for activations and weights, the two inputs to our core matrix multiplication operation.

FINE-GRAINED QUANTIZATION FOR ACTIVATIONS (INPUTS)

First, let's consider the activations, which are the input vectors to a given layer. An activation vector in a large model can be very long (e.g., with a dimension of 7168 in DeepSeek-V3). To quantize it, DeepSeek breaks it down into smaller, contiguous groups.

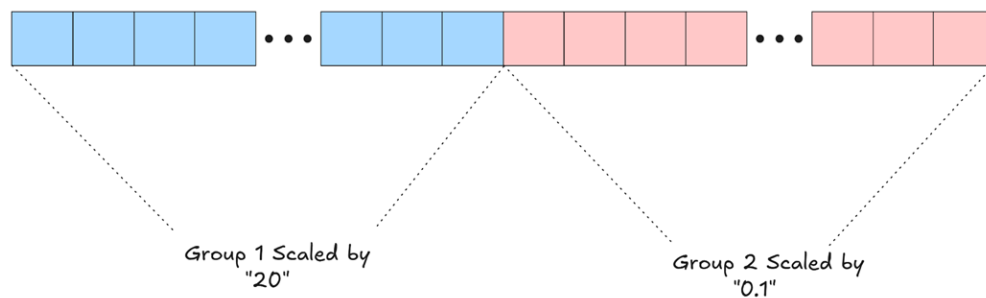


Figure 5.27 Fine-Grained Quantization for an activation vector. The vector is partitioned into smaller groups. Each group is scaled independently based on the maximum value *within that group*, preserving precision for values in groups that do not contain large outliers.

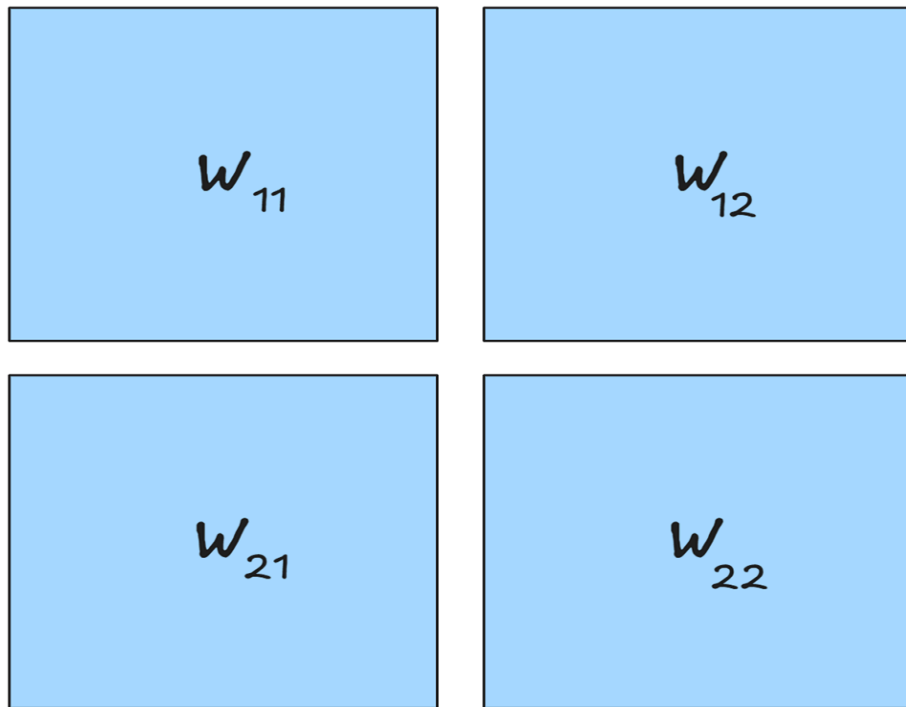
As figure 5.27 illustrates, an outlier in one group no longer affects the others. Let's say Group 1 contains values with a maximum of "20", while Group 2 contains much smaller values with a maximum of "0.1". With fine-grained quantization:

- All elements in Group 1 are scaled by 20.
- All elements in Group 2 are scaled by 0.1.

The small values in Group 2 are now scaled appropriately, preserving their relative differences and ensuring they are not "squashed" down to near-zero by the outlier in Group 1. This per-group scaling is crucial for maintaining the fidelity of the model's internal representations. In their implementation, DeepSeek uses a group size (N_c) of 128 elements for activations, meaning every 128 channels of an activation vector get their own private scaling factor.

FINE-GRAINED QUANTIZATION FOR WEIGHTS

A similar principle applies to the weight matrices, but adapted for their two-dimensional structure. A large weight matrix is not treated as one monolithic block. Instead, it is partitioned into smaller 2D "tiles" or "blocks."



W_{11} —————> Scaling Factor 1

W_{12} —————> Scaling Factor 2

W_{13} —————> Scaling Factor 3

W_{14} —————> Scaling Factor 4

Figure 5.28 Fine-Grained Quantization for a weight matrix. The matrix is divided into smaller blocks (e.g., W_{11} , W_{12} , etc.), and each block is quantized independently with its own unique scaling factor.

As shown in figure 5.28, a matrix W might be broken into four blocks: W_{11} , W_{12} , W_{21} , and W_{22} . Each of these blocks is quantized completely independently:

- W_{11} is scaled by **Scaling Factor 1**, based on its own internal max value.
- W_{12} is scaled by **Scaling Factor 2**, based on *its* max value.
- ...and so on for all blocks.

This block-wise approach for weights ensures that if a few parameters in one region of the matrix learn very large values, they do not degrade the precision of the millions of other weights in the remaining blocks. For their FP8 framework, DeepSeek uses a block size of 128×128 .

THE FULL MECHANISM IN ACTION

Now we can understand the complete flow diagram presented by the DeepSeek team. It visualizes how these two fine-grained inputs come together.

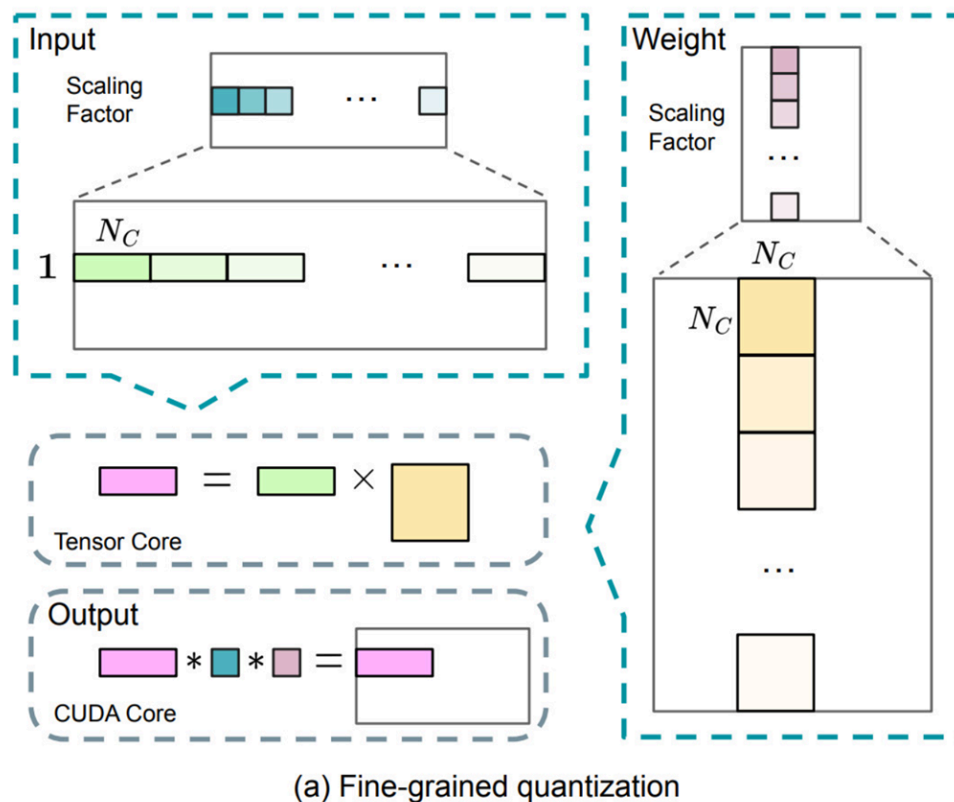


Figure 5.29 The complete Fine-Grained Quantization workflow.

1. **Input (Activations):** The input vector is shown on the top-left. It's partitioned into chunks of size N_c . A unique Scaling Factor (represented by the different shades of teal/green) is calculated for each chunk.
2. **Weight:** The weight matrix is shown on the top-right. It's partitioned into blocks of N_c (in this case, $N_c \times N_c$). Each block gets its own Scaling Factor (shades of pink/purple).
3. **Tensor Core Computation:** The core matrix multiplication ($\text{Output} = \text{Input} \times \text{Weight}$) is performed on the specialized, high-speed Tensor Cores. This operation takes the quantized FP8 values as input and produces a low-precision intermediate output (pink rectangle).
4. **CUDA Core De-quantization:** The final step happens on the general-purpose **CUDA Cores**. The low-precision output from the Tensor Core is de-quantized. This is done by multiplying it with the corresponding scaling factors from both the input and the weights to restore its original magnitude (albeit with some precision loss). This final, high-precision output is then ready for the next stage of the framework.

By breaking down the quantization process into fine-grained, independent chunks, DeepSeek ensures that the numerical precision is maintained at a local level, making the entire training process far more robust to the volatile and high-dynamic-range values that are common in LLMs. This simple "divide and conquer" strategy is one of the key enablers of stable FP8 training at scale.

5.5.8 Pillar 3: Increasing accumulation precision

The third pillar addresses a subtle but critical hardware limitation in modern GPUs. While GPUs are exceptionally good at performing low-precision matrix multiplications (like FP8 x FP8), there's a limit to the precision they use for the *intermediate* results *during* the multiplication process itself.

THE PROBLEM: LOSING PRECISION IN THE ACCUMULATOR

Let's revisit the matrix multiplication $Y = WX$. To compute a single element in the output matrix Y , we perform a dot product: we multiply corresponding elements from a row of W and a column of X , and then sum (or **accumulate**) all those products together.

$$Y_{ij} = W_{i1}X_{1j} + W_{i2}X_{2j} + \dots + W_{ik}X_{kj}$$

When W and X are in FP8, each individual product ($W_{in}X_{nj}$) is computed. The GPU's specialized hardware, called Tensor Cores, then needs to add these products up. The internal memory register that holds this running sum is called an accumulator.

The problem is that on modern hardware like NVIDIA's H800 GPUs, this internal accumulator has limited precision (e.g., around 14 bits), which is significantly lower than the standard 32-bit (FP32) precision. If the inner dimension k of the matrix multiplication is very large (e.g., 4096 or more, which is common in LLMs), we are summing up thousands of these small products. If the accumulator doesn't have enough precision, we can encounter underflow issues, where the intermediate sum is too small to be accurately represented, and the final result can have significant numerical errors, potentially as high as 2%. This can severely impact the model's accuracy.

THE DEEPSEEK SOLUTION: PROMOTION TO CUDA CORES

To solve this problem, DeepSeek implemented a strategy called promotion to CUDA Cores. The core idea is to leverage the two different types of computational units available on modern GPUs. Tensor Cores are highly specialized, designed to perform matrix multiplication at incredible speed but with limited precision. In contrast, CUDA Cores are the general-purpose workhorses of the GPU, capable of running a wide variety of tasks with full, high precision, albeit more slowly.

DeepSeek's strategy is to periodically move the intermediate accumulation results from the fast, low-precision Tensor Cores to the flexible, high-precision CUDA Cores, which can then perform the final accumulation in full FP32 precision.

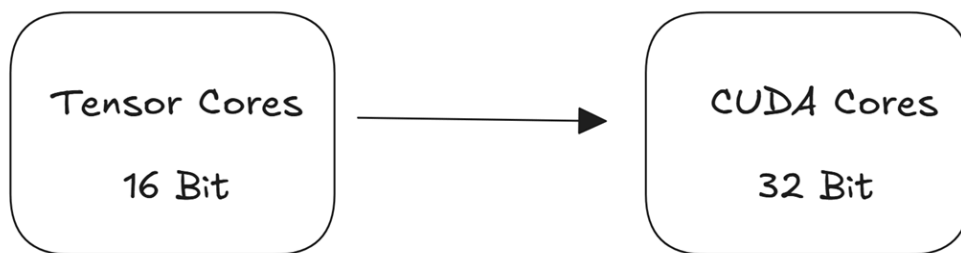


Figure 5.30 The core strategy for increasing accumulation precision: intermediate results are periodically moved from the low-precision environment of the Tensor Cores to the high-precision environment of the CUDA Cores.

To truly understand this mechanism, we need to look closer at the specific hardware components involved. The calculation begins within the Tensor Core, which performs high-speed, low-precision multiplications in small bursts using an instruction called WGMMA (Warp-Group-level Matrix Multiply-Accumulate).

During each WGMMA step, the intermediate results are accumulated in a low-precision register, labeled "Low Prec Acc" in the diagram (figure 5.30). The challenge is that this accumulator has limited precision. DeepSeek's innovation is to not wait until the entire calculation is finished. Instead, at a fixed interval (N_c), the partial sum from the "Low Prec Acc" is periodically "promoted" or transferred to a general-purpose CUDA Core.

The CUDA Core takes this partial sum and adds it to an "FP32 Register," which has full high-precision capabilities. By performing the final accumulation in this high-precision register, DeepSeek significantly reduces the risk of the underflow issues that plague standard low-precision accumulation, ensuring numerical stability.

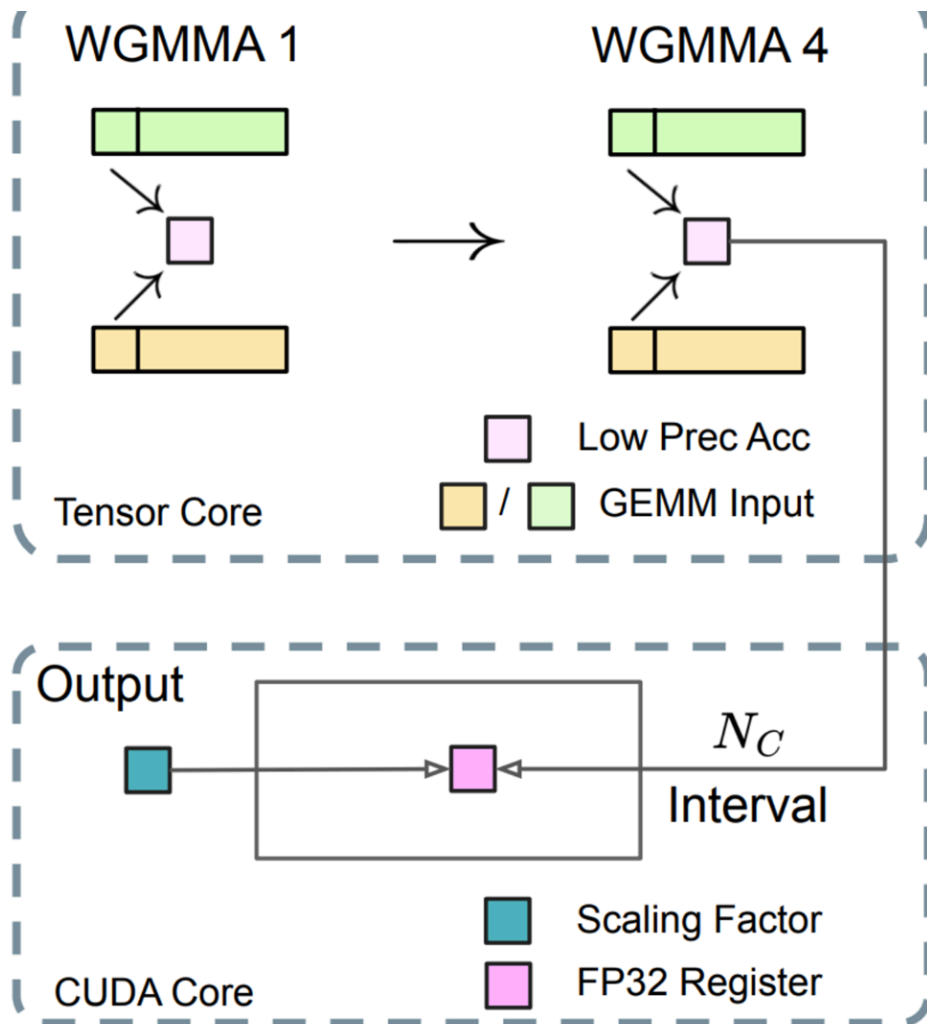


Figure 5.31 The Increasing Accumulation Precision mechanism in detail. Low-precision accumulation happens in bursts on the Tensor Core, and results are periodically promoted to a high-precision FP32 register on the CUDA Core.

Let's trace the journey of the data through this diagram step-by-step, defining every component:

PART 1: THE TENSOR CORE: HIGH-SPEED, LOW-PRECISION WORK

The top section of figure 5.31, labeled "Tensor Core," shows where the bulk of the raw computation happens.

1. **GEMM Input:** These are the two rectangular blocks at the very top representing the inputs to our matrix multiplication, $y = Wx$. One block represents the input tensor (e.g., the fine-grained activation vector), and the other represents the weight matrix. Both are already in the fast FP8 format.

2. **WGMMMA (Warp-Group-level Matrix Multiply-Accumulate):** This is a highly optimized, low-level NVIDIA instruction that tells a group of GPU threads (a "warp") to perform a chunk of a matrix multiplication. You can think of WGMMMA 1 through WGMMMA 4 as sequential steps in a much longer dot product. For instance, WGMMMA 1 might compute the sum of the first 32 products in our Y_{ij} equation.

3. **Low Prec Acc:** This is the small square register labeled "Low Prec Acc," representing the Low Precision Accumulator. It's an internal hardware register within the Tensor Core that holds the running sum. In WGMMMA 1, the first set of products is calculated and stored here. In the next step of the WGMMMA sequence, the next set of products would be calculated and added to this same accumulator. The key point is that this register has limited precision (around 14 bits).

PART 2: THE BRIDGE: PERIODIC PROMOTION TO HIGH PRECISION

The crucial innovation is the arrow that connects the Tensor Core to the CUDA Core. This is not a one-time data transfer at the end of the calculation; it is a periodic promotion.

Nc Interval: DeepSeek does not wait for all steps of the WGMMMA to complete. Instead, after a fixed interval of N_c element-wise operations (the paper specifies $N_c = 128$), the process is paused. The partial sum that has been calculated and stored in the "Low Prec Acc" register is copied out of the Tensor Core.

PART 3: THE CUDA CORE: LOW-SPEED, HIGH-PRECISION FINISHING

The bottom section of the diagram, labeled "CUDA Core," is where the final, numerically stable part of the process occurs. CUDA Cores are the GPU's general-purpose workhorses, capable of handling full FP32 operations.

FP32 Register (Large Pink Square): The partial sum from the Tensor Core arrives and is placed into a register that can store a full 32-bit floating-point number. This register has ample precision to hold the sum of thousands of products without any risk of underflow or precision loss. As more partial sums arrive every N_c interval from the Tensor Core, they are safely added to this high-precision running total.

Scaling Factor (Teal Block): At the same time, the CUDA Core can efficiently handle the de-quantization. It takes the Scaling Factors that were calculated during the Fine-Grained Quantization step (Pillar 2) and multiplies them with the high-precision accumulated value.

Output: The final result is a de-quantized, high-precision value that is numerically stable and ready to be stored as BF16 for the next layer in the network.

This hybrid approach perfectly embodies the "best of both worlds" philosophy. It uses the right tool for the right job fast, specialized Tensor Cores for the bulk of the multiplications and slower, general-purpose CUDA Cores for the final, high-precision accumulation and de-quantization. This synergy allows DeepSeek to achieve both the incredible speed of FP8 computation and the numerical accuracy of FP32 accumulation.

5.5.9 Pillar 4: Mantissa over exponents

As we discussed in section 5.5.3, any floating-point format is a trade-off between the number of bits allocated to the exponent (which determines the dynamic range) and the mantissa (which determines the precision).

For the 8-bit FP8 format, two main standards have emerged:

1. **E5M2:** Uses 5 bits for the exponent and 2 bits for the mantissa. This format has a larger dynamic range (it can represent a wider span of numbers) but lower precision.
2. **E4M3:** Uses 4 bits for the exponent and 3 bits for the mantissa. This format has a smaller dynamic range but higher precision.

THE CONVENTIONAL APPROACH

Prior to DeepSeek, a common strategy in mixed-precision training was to use a hybrid approach:

- Use the high-precision E4M3 for the forward pass (Fprop), where the values are more controlled.
- Use the high-range E5M2 for the backward pass (Dgrad and Wgrad), as gradients can sometimes have very large values (outliers) that could cause an overflow in the smaller range of E4M3.

DEEPSEEK IMPLEMENTATION: UNIFORM E4M3

The DeepSeek team argued that this hybrid approach was a workaround for a problem they had already solved. Thanks to their Fine-Grained Quantization (Pillar 2), the problem of outliers is significantly mitigated.

Because they are scaling activations and weights in small, independent blocks, a large outlier in one block does not affect the scaling of the others. This prevents the "squashing" of values that would normally lead to precision loss. The fine-grained scaling factors effectively manage the dynamic range at a local level.

This insight allowed them to make a powerful simplification: they chose to use the higher-precision E4M3 format uniformly for all operations, including both the forward and backward passes.

By relying on their fine-grained quantization to handle the dynamic range, they could consistently use the format with more mantissa bits, thereby preserving a higher level of precision throughout the entire training process. The DeepSeek paper notes:

"By operating on smaller element groups, our methodology effectively shares exponent bits among these grouped elements, mitigating the impact of the limited dynamic range."

This is a subtle but important innovation. It demonstrates how DeepSeek's different quantization techniques work together synergistically. The strength of their fine-grained scaling allowed them to make a more aggressive choice in favor of precision, which is a key contributor to the stability and performance of their FP8 training framework.

5.5.10 Pillar 5: Online quantization

The final piece of DeepSeek's quantization puzzle addresses the question of *when* the scaling factors are calculated. As we know, the scaling factor is derived from the maximum absolute value of a tensor. But which tensor? The one from the previous step, or the one we are currently processing?

THE CONVENTIONAL APPROACH: DELAYED QUANTIZATION

Many quantization frameworks use a technique called Delayed Quantization. In this approach, the scaling factor used to quantize the *current* tensor is derived from the maximum value observed in *past* iterations or batches. It maintains a running history of maximum values and uses that historical information to estimate a good scaling factor for the current step.

The problem with this approach is that the data distribution can change rapidly during training. The maximum value in the current batch might be significantly different from the historical maximum.

- If the current maximum is much larger than the historical one, using the old, smaller scaling factor can lead to overflow, where the quantized values exceed the representable range of FP8.
- If the current maximum is much smaller than the historical one, using the old, larger scaling factor can lead to underflow and a catastrophic loss of precision (the "squashing" problem we saw earlier).

THE DEEPSEEK SOLUTION: ONLINE QUANTIZATION

To solve this, DeepSeek uses Online Quantization. The idea is simple and robust: calculate the scaling factor in real-time, based on the data from the current tensor itself.

Instead of relying on historical information, the workflow is:

1. For the current batch of activations or weights, first perform a quick pass to find the maximum absolute value *within that specific batch*.
2. Use this "online" maximum value to derive the scaling factor.

3. Apply this fresh, perfectly calibrated scaling factor to quantize the current tensor.

This on-the-fly calculation ensures that the scaling factor is always perfectly tailored to the dynamic range of the data being processed at that exact moment. It completely avoids the risk of overflow or underflow caused by using stale, historical scaling factors.

While this adds a small computational overhead (the initial pass to find the maximum), it's worth noting that finding the maximum value in a tensor is an order of magnitude faster than the matrix multiplications that form the bulk of the computation. The benefit gained in terms of numerical stability and accuracy is therefore enormous. This commitment to using the most accurate, real-time information possible, even at the cost of a computationally inexpensive pre-pass, is a recurring theme in DeepSeek's design and is a key reason for the robustness of their FP8 training framework.

In the next chapter, we will apply this knowledge by creating a small-scale, DeepSeek-like model, demonstrating how these concepts come together in practice.

5.6 Summary

- Multi-Token Prediction (MTP) enhances data efficiency by training models to predict a horizon of future tokens simultaneously, providing richer gradient signals for long-range coherence.
- The MTP training objective implicitly assigns higher weight to "choice points" or critical pattern-shifting tokens in a sequence, thereby developing better planning capabilities in the model.
- DeepSeek's causal MTP implementation sequentially refines a hidden state, where the prediction for one token informs the next, creating a powerful forecasting mechanism.
- Quantization is essential for large-scale training, reducing memory usage and computational cost by converting high-precision parameters to low-precision formats like FP8.
- DeepSeek's FP8 framework uses a mixed precision strategy, performing core computations in low precision while storing sensitive components, such as master weights and optimizer states, in high precision to ensure numerical stability.
- **Fine-Grained Quantization:** To mitigate precision loss from outliers, DeepSeek calculates separate scaling factors for small blocks of activations and weights, preserving fidelity at a local level.
- **Increasing Accumulation Precision:** DeepSeek promotes intermediate results from low-precision Tensor Cores to high-precision CUDA Cores, where accumulation occurs in full FP32 precision to prevent underflow errors.
- **Mantissa over Exponents:** DeepSeek uniformly uses the higher-precision E4M3 format for both forward and backward passes, enabled by fine-grained quantization, which effectively manages dynamic range.

- **Online Quantization:** DeepSeek calculates scaling factors in real-time based on the current batch's data, ensuring accurate scaling and avoiding the instability associated with historical data.